

30 requêtes SQL piégées, corrigées



Le corrigé de référence pour un test technique Data Analyst, sur un seul schéma e-commerce du début à la fin.

Un test technique SQL ne piège pas sur la syntaxe : il piège sur le `WHERE` qui annule un `LEFT JOIN`, le `NOT IN` qui rencontre un `NULL`, la fenêtre sans `PARTITION BY`. Ce guide reprend 30 requêtes construites sur les mêmes tables (clients, commandes, produits) pour que la progression se lise comme un vrai test, avec à chaque fois le piège exact et l'erreur qui fait échouer un junior.

Ce que 30 tests techniques SQL m'ont appris

Sur les tests techniques SQL que j'ai vus passer, les candidats qui échouent ne butent presque jamais sur la syntaxe pure. Ils butent sur un `LEFT JOIN` piégé par un `WHERE` mal placé, un `NOT IN` qui croise un `NULL`, un cumul de fenêtre posé sans `PARTITION BY`. Des erreurs invisibles à la lecture, qui ne provoquent aucun message rouge, juste un chiffre faux. Les 30 requêtes qui suivent utilisent un seul schéma e-commerce, du début à la fin.

```
-- Schéma de référence (1/2) : clients, catalogue, commandes
CREATE TABLE clients (
  client_id      serial PRIMARY KEY,
  nom            text NOT NULL,
  ville         text,
  email         text,
  date_inscription date
);

CREATE TABLE categories (
  categorie_id    serial PRIMARY KEY,
  nom            text NOT NULL,
  parent_categorie_id integer REFERENCES categories(categorie_id)
);

CREATE TABLE produits (
  produit_id     serial PRIMARY KEY,
  nom           text NOT NULL,
  categorie_id   integer REFERENCES categories(categorie_id),
  prix          numeric(10,2) NOT NULL
);

CREATE TABLE commandes (
  commande_id    serial PRIMARY KEY,
  client_id      integer REFERENCES clients(client_id),
  date_commande  timestamp NOT NULL,
  statut         text NOT NULL, -- 'validee' | 'remboursee' | 'annulee'
  montant_total  numeric(10,2)
);
```

```
-- Schéma de référence (2/2) : détail, prospects, archive
CREATE TABLE lignes_commande (
  ligne_id      serial PRIMARY KEY,
  commande_id   integer REFERENCES commandes(commande_id),
  produit_id    integer REFERENCES produits(produit_id),
  quantite      integer NOT NULL,
  prix_unitaire numeric(10,2) NOT NULL
);

CREATE TABLE prospects (
  prospect_id serial PRIMARY KEY,
  nom          text,
  email        text,
  source       text
);

CREATE TABLE commandes_archive (
  commande_id   integer PRIMARY KEY,
  client_id     integer REFERENCES clients(client_id),
  date_commande timestamp NOT NULL,
  statut        text NOT NULL,
  montant_total numeric(10,2)
);
```

commandes_archive reprend les commandes migrées d'un ancien système, même structure que commandes : elle ne sert qu'à la requête 28. Garde ces deux blocs sous les yeux, ils ne changent jamais.

Filtrer et trier sans se planter

C'est la première chose testée. Un WHERE, un ORDER BY, un alias mal posés annoncent déjà le niveau du candidat.

1. Lister les clients d'une ville (WHERE)

Énoncé : Le recruteur demande la liste des clients de Lyon.

```
SELECT client_id, nom, ville
FROM clients
WHERE ville = 'Lyon';
```

Le piège : une égalité stricte sur du texte libre est sensible à la casse et aux espaces : elle rate 'LYON', 'lyon' ou 'Lyon' saisis autrement dans le formulaire d'inscription.

```
SELECT client_id, nom, ville
FROM clients
WHERE TRIM(ville) ILIKE 'lyon';
```

Erreur junior : comparer avec = sur un champ texte libre, au lieu de combiner ILIKE et TRIM pour neutraliser à la fois la casse et les espaces parasites.

2. Trier les commandes par montant (ORDER BY) PIÈGE CLASSIQUE

Énoncé : Afficher le top 10 des commandes par montant décroissant.

```
SELECT commande_id, client_id, montant_total
FROM commandes
ORDER BY montant_total DESC
LIMIT 10;
```

Le piège : sur PostgreSQL, un ORDER BY ... DESC place par défaut les valeurs NULL en premier. Les commandes sans montant renseigné remontent en tête d'un classement censé montrer les plus grosses ventes.

```
SELECT commande_id, client_id, montant_total
FROM commandes
ORDER BY montant_total DESC NULLS LAST
LIMIT 10;
```

Erreur junior : oublier NULLS LAST, en supposant que DESC place naturellement les NULL à la fin comme dans d'autres moteurs.

3. Filtrer sur une plage de dates (BETWEEN) PIÈGE CLASSIQUE

Énoncé : Extraire toutes les commandes de janvier 2026.

```
SELECT commande_id, client_id, date_commande
FROM commandes
WHERE date_commande BETWEEN '2026-01-01' AND '2026-01-31';
```

Le piège : date_commande est un timestamp. La borne '2026-01-31' vaut minuit pile : toute commande passée le 31 janvier après 00h00 est exclue du résultat, sans aucune erreur visible.

```
SELECT commande_id, client_id, date_commande
FROM commandes
WHERE date_commande >= '2026-01-01'
      AND date_commande < '2026-02-01';
```

Erreur junior : utiliser BETWEEN avec des bornes de date sur une colonne timestamp, en oubliant que la borne haute est une heure précise, pas une journée entière.

À retenir avant tout WHERE

Trois réflexes qui évitent une bonne partie des faux résultats d'un test technique.

Casse du texte

ILIKE ou normalise, jamais une égalité stricte sur du texte libre.

Tri et NULL

Un ORDER BY sur une colonne à trous exige NULLS FIRST ou NULLS LAST explicite.

Bornes de dates

Sur un timestamp, préfère >= début AND < fin plutôt qu'un BETWEEN qui coupe la journée.

4. Compter les commandes distinctes par client (COUNT DISTINCT) PIÈGE CLASSIQUE

Énoncé : Compter le nombre de commandes par client, à partir du détail des lignes produit.

```
SELECT c.client_id, COUNT(lc.ligne_id) AS nb_commandes
FROM commandes c
JOIN lignes_commande lc ON lc.commande_id = c.commande_id
GROUP BY c.client_id;
```

Le piège : la jointure avec lignes_commande éclate chaque commande en autant de lignes que de produits achetés. COUNT(lc.ligne_id) compte des lignes de produit, pas des commandes.

```
SELECT c.client_id, COUNT(DISTINCT c.commande_id) AS nb_commandes
FROM commandes c
JOIN lignes_commande lc ON lc.commande_id = c.commande_id
GROUP BY c.client_id;
```

Erreur junior : compter une colonne après une jointure qui multiplie les lignes, sans DISTINCT sur la clé qu'on veut vraiment compter.

5. Calculer une part en pourcentage (CAST)

Énoncé : Afficher la part de chaque produit dans la quantité totale vendue.

```
SELECT produit_id,  
       SUM(quantite) * 100 / (SELECT SUM(quantite) FROM lignes_commande)  
       AS part_pct  
FROM lignes_commande  
GROUP BY produit_id;
```

Le piège : SUM(quantite) renvoie un entier. Une division entier / entier tronque le résultat vers zéro : un produit qui pèse 4,3 % du volume affiche 4, pas 4,3, la partie décimale disparaît silencieusement avant même que le CAST n'entre en jeu.

```
SELECT produit_id,  
       ROUND(SUM(quantite)::numeric * 100  
             / (SELECT SUM(quantite) FROM lignes_commande), 1) AS part_pct  
FROM lignes_commande  
GROUP BY produit_id;
```

Erreur junior : caster après avoir divisé au lieu de caster avant, en pensant que ::numeric répare une division déjà tronquée.

Les jointures, terrain où les juniors se trahissent

Un INNER, un LEFT et un self-join suffisent à savoir si tu comprends vraiment ce que fait une jointure, pas seulement sa syntaxe.

6. Clients sans commande (LEFT JOIN) ÉLIMINATOIRE

Énoncé : Lister les clients qui n'ont jamais passé de commande validée.

```
SELECT cl.client_id, cl.nom  
FROM clients cl  
LEFT JOIN commandes co ON co.client_id = cl.client_id  
WHERE co.commande_id IS NULL  
      AND co.statut = 'validee';
```

Le piège : co.statut est NULL sur les lignes générées par le LEFT JOIN. NULL = 'validee' n'est ni vrai ni faux : la ligne est écartée. Le résultat est vide, alors que des clients sans commande validée existent bien.

```
SELECT cl.client_id, cl.nom  
FROM clients cl  
LEFT JOIN commandes co  
      ON co.client_id = cl.client_id AND co.statut = 'validee'  
WHERE co.commande_id IS NULL;
```

Erreur junior : ajouter la condition sur le statut de la commande dans le WHERE au lieu du ON, ce qui annule les lignes NULL que le LEFT JOIN venait de produire.

7. Chiffre d'affaires des commandes validées (JOIN) PIÈGE CLASSIQUE

Énoncé : Calculer le CA total des commandes validées.

```
SELECT SUM(co.montant_total) AS ca_total
FROM commandes co
JOIN lignes_commande lc ON lc.commande_id = co.commande_id
WHERE co.statut = 'validee';
```

Le piège : une commande de 3 produits a 3 lignes dans `lignes_commande`. Le `JOIN` duplique la ligne commande 3 fois, et `montant_total` est donc sommé 3 fois : le CA affiché est mécaniquement gonflé.

```
SELECT SUM(co.montant_total) AS ca_total
FROM commandes co
WHERE co.statut = 'validee'
AND EXISTS (
    SELECT 1 FROM lignes_commande lc WHERE lc.commande_id = co.commande_id
);
```

Erreur junior : sommer une colonne de la table 'un' après un `JOIN` vers la table 'plusieurs', sans dédupliquer la commande avant de sommer.

8. Jointure sur la mauvaise clé (colonnes homonymes) PIÈGE CLASSIQUE

Énoncé : Afficher chaque ligne de commande avec le nom et le prix du produit associé.

```
SELECT lc.ligne_id, p.nom, p.prix
FROM lignes_commande lc, produits p
WHERE lc.commande_id = p.produit_id;
```

Le piège : `commande_id` et `produit_id` sont deux identifiants sans rapport, mais numériquement ils se recoupent parfois par coïncidence. La requête s'exécute sans erreur et renvoie un résultat plausible mais faux.

```
SELECT lc.ligne_id, p.nom, p.prix
FROM lignes_commande lc
JOIN produits p ON p.produit_id = lc.produit_id;
```

Erreur junior : joindre deux colonnes id qui portent presque le même nom sans vérifier qu'elles désignent le même référentiel.

Trois pièges de jointure, trois causes différentes : un NULL non filtré, une cardinalité qui gonfle un agrégat, une clé homonyme mal choisie. Le quatrième vient d'un vieux réflexe : le self-join sans identifiants ordonnés.

9. Comparer deux commandes du même client (self-join) PIÈGE CLASSIQUE

Énoncé : Détecter les paires de commandes passées le même jour par un même client, signe possible de doublon.

```
SELECT c1.commande_id, c2.commande_id, c1.client_id
FROM commandes c1, commandes c2
WHERE c1.client_id = c2.client_id
AND c1.date_commande::date = c2.date_commande::date;
```

Le piège : sans condition sur les identifiants, chaque commande est comparée à elle-même (une paire inutile) et chaque vraie paire apparaît deux fois, dans les deux sens.

```
SELECT c1.commande_id AS commande_a, c2.commande_id AS commande_b, c1.client_id
FROM commandes c1
JOIN commandes c2
ON c1.client_id = c2.client_id AND c1.commande_id < c2.commande_id
WHERE c1.date_commande::date = c2.date_commande::date;
```

Erreur junior : oublier, sur un self-join, la condition qui ordonne les deux identifiants de commande : elle seule évite à la fois l'auto-comparaison et les doublons de chaque paire.

LEFT JOIN et WHERE, le piège numéro un des tests SQL

Une condition sur la table de droite ne se met jamais au même endroit selon ce qu'elle doit faire.

Filtre dans le ON

Tu restreins ce qui peut matcher à droite, mais tu gardes toutes les lignes de gauche sans correspondance.

Filtre dans le WHERE

Tu filtres après la jointure : les lignes sans correspondance disparaissent, le LEFT JOIN se comporte comme un INNER JOIN.

10. Jointure à trois tables (clients, commandes, produits)

Énoncé : Lister chaque commande avec le nom du client et celui du produit acheté.

```
SELECT cl.nom, co.commande_id, p.nom AS produit
FROM clients cl
JOIN commandes co ON co.client_id = cl.client_id
JOIN lignes_commande lc ON lc.commande_id = co.commande_id
JOIN produits p ON p.produit_id = lc.produit_id;
```

Le piège : si une ligne de commande a un produit_id NULL (produit retiré du catalogue), le dernier INNER JOIN fait disparaître toute la ligne, y compris la commande et le client, du rapport.

```
SELECT cl.nom, co.commande_id, p.nom AS produit
FROM clients cl
JOIN commandes co ON co.client_id = cl.client_id
JOIN lignes_commande lc ON lc.commande_id = co.commande_id
LEFT JOIN produits p ON p.produit_id = lc.produit_id;
```

Erreur junior : enchaîner des INNER JOIN sur toute la chaîne clients/commandes/produits, ce qui élimine silencieusement les lignes dont le produit n'est plus référencé.

11. Rapprocher clients et prospects (FULL OUTER JOIN) PIÈGE CLASSIQUE

Énoncé : Construire une liste unique de contacts à partir des clients et des prospects, rapprochés par email.

```
SELECT cl.email, cl.nom, pr.source
FROM clients cl
FULL OUTER JOIN prospects pr ON pr.email = cl.email;
```

Le piège : cl.email est NULL sur toutes les lignes qui n'existent que côté prospects. Ces contacts remontent dans le rapport avec une colonne email vide, alors que l'email existe bien côté prospects.

```
SELECT COALESCE(cl.email, pr.email) AS email,
       cl.nom AS nom_client,
       pr.source AS source_prospect
FROM clients cl
FULL OUTER JOIN prospects pr ON pr.email = cl.email;
```

Erreur junior : sélectionner la colonne d'un seul côté après un FULL OUTER JOIN, au lieu de fusionner les deux côtés avec COALESCE.

Vérifier la cardinalité avant de faire confiance à une jointure

Une jointure qui multiplie les lignes ne provoque jamais d'erreur SQL, seulement un chiffre faux.

Grain

Une ligne du résultat représente quoi, exactement?

Cardinalité

La clé de jointure est-elle unique du côté droit?

Contrôle

Compare le nombre de lignes avant et après la jointure.

12. Produits jamais commandés (NOT EXISTS) ÉLIMINATOIRE

Énoncé : Trouver les produits du catalogue qui n'ont jamais été commandés.

```
SELECT p.produit_id, p.nom
FROM produits p
WHERE p.produit_id NOT IN (SELECT produit_id FROM lignes_commande);
```

Le piège : Si `lignes_commande.produit_id` contient ne serait-ce qu'une valeur NULL, `NOT IN` renvoie systématiquement un résultat vide pour toute la requête, sans le moindre message d'erreur.

```
SELECT p.produit_id, p.nom
FROM produits p
WHERE NOT EXISTS (
    SELECT 1 FROM lignes_commande lc WHERE lc.produit_id = p.produit_id
);
```

Erreur junior : utiliser `NOT IN` avec une sous-requête qui peut contenir un NULL, au lieu de `NOT EXISTS` qui ne connaît pas ce piège.

Les agrégations, le piège qui passe sous le radar

GROUP BY, HAVING et CASE WHEN produisent des chiffres qui semblent justes, jusqu'à ce qu'on regarde d'où ils viennent.

13. Chiffre d'affaires par mois (DATE_TRUNC) PIÈGE CLASSIQUE

Énoncé : Calculer le CA validé mois par mois, sur plusieurs années d'historique.

```
SELECT EXTRACT(MONTH FROM date_commande) AS mois, SUM(montant_total) AS ca
FROM commandes
WHERE statut = 'validee'
GROUP BY EXTRACT(MONTH FROM date_commande);
```

Le piège : `EXTRACT(MONTH ...)` seul fusionne janvier 2025 et janvier 2026 dans le même groupe 'mois = 1' : deux années différentes s'additionnent dans un seul total.

```
SELECT DATE_TRUNC('month', date_commande) AS mois, SUM(montant_total) AS ca
FROM commandes
WHERE statut = 'validee'
GROUP BY DATE_TRUNC('month', date_commande)
ORDER BY mois;
```

Erreur junior : grouper sur `EXTRACT(MONTH ...)` au lieu de `DATE_TRUNC('month', ...)`, en oubliant que le mois seul ne distingue pas les années.

14. Clients avec plus de 3 commandes (HAVING) ÉLIMINATOIRE

Énoncé : Lister les clients qui ont passé plus de 3 commandes.

```
SELECT client_id, COUNT(*) AS nb
FROM commandes
WHERE COUNT(*) > 3
GROUP BY client_id;
```

Le piège : un agrégat comme COUNT(*) ne peut pas être évalué dans un WHERE : le WHERE filtre les lignes avant tout regroupement. PostgreSQL refuse la requête à la compilation.

```
SELECT client_id, COUNT(*) AS nb
FROM commandes
GROUP BY client_id
HAVING COUNT(*) > 3;
```

Erreur junior : écrire le filtre sur un agrégat dans le WHERE au lieu du HAVING, une confusion sur l'ordre logique d'exécution qui casse la requête net.

Un agrégat correct commence par un WHERE propre

AVG, SUM et COUNT ne savent pas ce qu'un statut annulé veut dire pour toi.

Avant d'écrire un GROUP BY, pose-toi la question du périmètre : quelles lignes ont le droit d'entrer dans le calcul? Un agrégat sur des données non filtrées reste juste en SQL et faux en business.

15. Panier moyen hors remboursements (AVG) PIÈGE CLASSIQUE

Énoncé : Calculer le panier moyen réel des clients.

```
SELECT AVG(montant_total) AS panier_moyen
FROM commandes;
```

Le piège : cette moyenne inclut les commandes annulées et remboursées : des montants qui n'ont jamais généré de revenu réel gonflent silencieusement le panier moyen affiché.

```
SELECT AVG(montant_total) AS panier_moyen
FROM commandes
WHERE statut = 'validee';
```

Erreur junior : calculer une moyenne sur toutes les commandes sans exclure les statuts annulée et remboursée.

16. Top 3 produits par catégorie (ROW_NUMBER) PIÈGE CLASSIQUE

Énoncé : Afficher les 3 produits les plus vendus de chaque catégorie.

```
SELECT p.produit_id, p.nom, p.categorie_id, SUM(lc.quantite) AS qte_vendue
FROM produits p
JOIN lignes_commande lc ON lc.produit_id = p.produit_id
GROUP BY p.produit_id, p.nom, p.categorie_id
ORDER BY qte_vendue DESC
LIMIT 3;
```

Le piège : un LIMIT global ne garde que les 3 meilleurs produits toutes catégories confondues. Les catégories moins vendues, même avec de bons produits, disparaissent entièrement du résultat.


```

SELECT produit_id, nom, categorie_id, qte_vendue
FROM (
  SELECT p.produit_id, p.nom, p.categorie_id, SUM(lc.quantite) AS qte_vendue,
         ROW_NUMBER() OVER (PARTITION BY p.categorie_id
                           ORDER BY SUM(lc.quantite) DESC) AS rang
  FROM produits p
  JOIN lignes_commande lc ON lc.produit_id = p.produit_id
  GROUP BY p.produit_id, p.nom, p.categorie_id
) classement
WHERE rang <= 3;

```

Erreur junior : appliquer un LIMIT global au lieu d'un classement par groupe avec ROW_NUMBER et PARTITION BY.

Le LIMIT mal posé de la requête 16 a un cousin direct : un GROUP BY qui inclut la mauvaise colonne peut, lui aussi, rendre une recherche de doublons structurellement aveugle.

17. Détecter des doublons de commande (HAVING COUNT) PIÈGE CLASSIQUE

Énoncé : Repérer les commandes qui auraient été enregistrées deux fois par erreur.

```

SELECT commande_id, client_id, date_commande, montant_total, COUNT(*)
FROM commandes
GROUP BY commande_id, client_id, date_commande, montant_total
HAVING COUNT(*) > 1;

```

Le piège : commande_id est la clé primaire : chaque groupe ne contient jamais qu'une seule ligne par construction. Cette requête ne trouvera jamais aucun doublon, même s'il y en a.

```

SELECT client_id, date_commande, montant_total, COUNT(*) AS occurrences
FROM commandes
GROUP BY client_id, date_commande, montant_total
HAVING COUNT(*) > 1;

```

Erreur junior : inclure la clé technique unique dans le GROUP BY qui doit détecter des doublons métier, ce qui rend le test toujours faux.

18. Somme conditionnelle avec CASE WHEN

Énoncé : Calculer, par client, le CA validé, le CA remboursé, et leur différence.

```

SELECT client_id,
       SUM(CASE WHEN statut = 'validee' THEN montant_total END) AS ca_valide,
       SUM(CASE WHEN statut = 'remboursee' THEN montant_total END) AS ca_rembourse,
       SUM(CASE WHEN statut = 'validee' THEN montant_total END)
       - SUM(CASE WHEN statut = 'remboursee' THEN montant_total END) AS solde
FROM commandes
GROUP BY client_id;

```

Le piège : un CASE WHEN sans ELSE renvoie NULL quand aucune ligne ne correspond. Pour un client sans aucun remboursement, ca_rembourse vaut NULL, et solde = ca_valide - NULL redevient NULL.

```

SELECT client_id,
       SUM(CASE WHEN statut = 'validee' THEN montant_total ELSE 0 END) AS ca_valide,
       SUM(CASE WHEN statut = 'remboursee' THEN montant_total ELSE 0 END) AS ca_rembourse,
       SUM(CASE WHEN statut = 'validee' THEN montant_total ELSE 0 END)
       - SUM(CASE WHEN statut = 'remboursee' THEN montant_total ELSE 0 END) AS solde
FROM commandes
GROUP BY client_id;

```

Erreur junior : oublier ELSE 0 dans un CASE WHEN utilisé dans un calcul arithmétique : le NULL se propage et fait disparaître un solde pourtant valide.

Sous-requêtes et CTE, la lisibilité qui trahit le niveau

Ici, ce n'est plus la syntaxe qui compte : c'est la capacité à écrire une requête qu'un collègue peut relire six mois plus tard.

19. Clients au-dessus de la moyenne (sous-requête scalaire) PIÈGE CLASSIQUE

Énoncé : Lister les commandes dont le montant dépasse la moyenne générale des commandes.

```
SELECT co.commande_id, co.client_id, co.montant_total
FROM commandes co
WHERE co.montant_total > (
    SELECT AVG(c2.montant_total) FROM commandes c2 WHERE c2.client_id = co.client_id
);
```

Le piège : la sous-requête est corrélée au client de la ligne, elle recalcule donc la moyenne propre à chaque client. Ce n'est plus 'au-dessus de la moyenne générale', mais 'au-dessus de sa propre moyenne', un résultat différent de la demande.

```
SELECT co.commande_id, co.client_id, co.montant_total
FROM commandes co
WHERE co.montant_total > (
    SELECT AVG(montant_total) FROM commandes
);
```

Erreur junior : corrélér par réflexe une sous-requête à la ligne externe, alors que la demande porte sur une moyenne générale, pas une moyenne par client.

20. Dernière commande de chaque client (sous-requête corrélée) ÉLIMINATOIRE

Énoncé : Afficher, pour chaque client, la date de sa dernière commande.

```
SELECT cl.client_id, cl.nom,
    (SELECT co.date_commande FROM commandes co
     WHERE co.client_id = cl.client_id) AS derniere_commande
FROM clients cl;
```

Le piège : dès qu'un client a plus d'une commande, la sous-requête en SELECT renvoie plusieurs lignes. PostgreSQL arrête l'exécution avec une erreur : plus d'une ligne renvoyée par une sous-requête utilisée comme expression.

```
SELECT cl.client_id, cl.nom,
    (SELECT MAX(co.date_commande) FROM commandes co
     WHERE co.client_id = cl.client_id) AS derniere_commande
FROM clients cl;
```

Erreur junior : écrire une sous-requête corrélée en SELECT sans agrégat, alors qu'elle doit renvoyer une valeur unique par ligne externe.

21. Isoler les commandes suspectes (WITH)

Énoncé : Compter par ville les commandes de plus de 500 euros, en isolant volontairement ce calcul du reste de la requête.

```
WITH commandes_suspectes AS (
  SELECT co.*, cl.ville
  FROM commandes co
  JOIN clients cl ON cl.client_id = co.client_id
  WHERE co.montant_total > 500
)
SELECT ville, COUNT(*) AS nb_suspectes
FROM commandes_suspectes
GROUP BY ville;
```

Le piège : cette CTE n'est référencée qu'une seule fois. Depuis PostgreSQL 12, une CTE non récursive référencée une seule fois n'est plus une frontière d'optimisation par défaut : le planificateur la replie (inline) dans la requête principale, comme une sous-requête ordinaire. Le junior qui écrit ce WITH pour figer un calcul intermédiaire perd cette garantie, sans erreur ni avertissement.

```
WITH commandes_suspectes AS MATERIALIZED (
  SELECT co.*, cl.ville
  FROM commandes co
  JOIN clients cl ON cl.client_id = co.client_id
  WHERE co.montant_total > 500
)
SELECT ville, COUNT(*) AS nb_suspectes
FROM commandes_suspectes
GROUP BY ville;
```

Erreur junior : croire qu'une CTE reste toujours une étape calculée à part, alors que depuis PostgreSQL 12 une CTE référencée une seule fois est repliée par défaut dans la requête principale, sauf si MATERIALIZED force l'isolement.

Une sous-requête doit rester lisible par quelqu'un d'autre

En entretien, une requête illisible envoie le même signal qu'une requête fausse.

Si une sous-requête corrélée s'imbrique sur plus de deux niveaux, ou si tu ne peux pas expliquer à voix haute pourquoi elle est corrélée, passe par une CTE nommée. La lisibilité n'est pas un supplément d'âme, c'est ce qui prouve que tu comprends ta propre requête.

22. Hiérarchie de catégories (WITH RECURSIVE) ÉLIMINATOIRE

Énoncé : Construire l'arborescence complète des catégories, de la racine jusqu'aux sous-catégories.

```
WITH RECURSIVE arbre AS (
  SELECT categorie_id, nom, parent_categorie_id, 1 AS niveau
  FROM categories
  WHERE parent_categorie_id IS NULL
  UNION ALL
  SELECT c.categorie_id, c.nom, c.parent_categorie_id, a.niveau + 1
  FROM categories c
  JOIN arbre a ON c.categorie_id = a.categorie_id
)
SELECT * FROM arbre;
```

Le piège : la jointure récursive relie chaque catégorie à elle-même (`categorie_id = categorie_id`) au lieu de descendre vers ses enfants. La récursion ne converge jamais : c'est une boucle infinie en plein test technique.

```
WITH RECURSIVE arbre AS (
  SELECT categorie_id, nom, parent_categorie_id, 1 AS niveau
  FROM categories
  WHERE parent_categorie_id IS NULL
  UNION ALL
  SELECT c.categorie_id, c.nom, c.parent_categorie_id, a.niveau + 1
  FROM categories c
  JOIN arbre a ON c.parent_categorie_id = a.categorie_id
)
SELECT * FROM arbre;
```

Erreur junior : écrire la condition de récursion sur le mauvais couple de colonnes, au lieu de relier `parent_categorie_id` de l'enfant à `categorie_id` du parent.

23. Colonne calculée par sous-requête (SELECT imbriqué) PIÈGE CLASSIQUE

Énoncé : Afficher, à côté de chaque client, son chiffre d'affaires personnel.

```
SELECT cl.client_id, cl.nom,
       (SELECT SUM(co.montant_total) FROM commandes co) AS ca_client
FROM clients cl;
```

Le piège : la sous-requête ne filtre pas sur le client de la ligne externe : elle calcule le total général et l'affiche, identique, sur chaque ligne du résultat.

```
SELECT cl.client_id, cl.nom,
       (SELECT SUM(co.montant_total) FROM commandes co
        WHERE co.client_id = cl.client_id) AS ca_client
FROM clients cl;
```

Erreur junior : oublier de corréler la sous-requête au client de la ligne externe, ce qui affiche le même total pour tout le monde.

Fenêtrage et requêtes piège, le niveau qui différencie

Les dernières requêtes combinent plusieurs pièges à la fois, exactement comme la dernière question d'un vrai test technique.

24. Classer les clients par CA (RANK) PIÈGE CLASSIQUE

Énoncé : Classer les clients par chiffre d'affaires validé, pour communiquer un top 3 au métier.

```
SELECT cl.client_id, cl.nom, SUM(co.montant_total) AS ca,
       ROW_NUMBER() OVER (ORDER BY SUM(co.montant_total) DESC) AS rang
FROM clients cl
JOIN commandes co ON co.client_id = cl.client_id
WHERE co.statut = 'validee'
GROUP BY cl.client_id, cl.nom;
```

Le piège : `ROW_NUMBER` attribue un rang unique à chaque ligne, même en cas d'égalité parfaite. Deux clients au même CA se retrouvent classés 1 et 2, alors qu'ils sont à égalité.

```
SELECT cl.client_id, cl.nom, SUM(co.montant_total) AS ca,
       RANK() OVER (ORDER BY SUM(co.montant_total) DESC) AS rang
FROM clients cl
JOIN commandes co ON co.client_id = cl.client_id
WHERE co.statut = 'validee'
GROUP BY cl.client_id, cl.nom;
```

Erreur junior : utiliser ROW_NUMBER pour un classement communiqué au métier, alors que RANK gère correctement les ex-aequo.

25. Cumul du CA dans le temps (SUM OVER) PIÈGE CLASSIQUE

Énoncé : Afficher, pour chaque client, le cumul de son CA commande après commande.

```
SELECT client_id, date_commande, montant_total,
       SUM(montant_total) OVER (PARTITION BY client_id) AS cumul
FROM commandes
WHERE statut = 'validee';
```

Le piège : sans ORDER BY dans le OVER, la fenêtre couvre tout le partitionnement d'un coup : cumul affiche le total fixe du client, identique sur chaque ligne, pas un cumul progressif.

```
SELECT client_id, date_commande, montant_total,
       SUM(montant_total) OVER (PARTITION BY client_id
                               ORDER BY date_commande) AS cumul
FROM commandes
WHERE statut = 'validee';
```

Erreur junior : omettre l'ORDER BY dans la clause OVER, en pensant que PARTITION BY suffit à produire un cumul progressif.

26. Variation vs mois précédent (LAG) PIÈGE CLASSIQUE

Énoncé : Comparer le CA mensuel de chaque client à celui du mois précédent.

```
WITH ca_mensuel AS (
  SELECT client_id, DATE_TRUNC('month', date_commande) AS mois,
         SUM(montant_total) AS ca
  FROM commandes
  WHERE statut = 'validee'
  GROUP BY client_id, DATE_TRUNC('month', date_commande)
)
SELECT client_id, mois, ca,
       ca - LAG(ca) OVER (ORDER BY mois) AS variation
FROM ca_mensuel;
```

Le piège : sans PARTITION BY client_id, LAG regarde la ligne précédente dans l'ordre global des mois, toutes personnes confondues : la variation d'un client peut être comparée au CA d'un autre client.

```
WITH ca_mensuel AS (
  SELECT client_id, DATE_TRUNC('month', date_commande) AS mois,
         SUM(montant_total) AS ca
  FROM commandes
  WHERE statut = 'validee'
  GROUP BY client_id, DATE_TRUNC('month', date_commande)
)
SELECT client_id, mois, ca,
       ca - LAG(ca) OVER (PARTITION BY client_id ORDER BY mois) AS variation
FROM ca_mensuel;
```

Erreur junior : oublier `PARTITION BY client_id` dans `LAG`, ce qui compare des mois de clients différents dès que leurs périodes se chevauchent.

Une window function sans `PARTITION BY` explicite est suspecte

`PARTITION BY` délimite le périmètre de calcul; l'oublier mélange des groupes qui n'ont rien à voir.

Avec `PARTITION BY`

Le calcul (rang, cumul, `LAG`) reste confiné à chaque client, chaque catégorie, chaque groupe métier.

Sans `PARTITION BY`

La fenêtre couvre tout le jeu de résultat : rangs faux, cumuls qui ne veulent rien dire, comparaisons entre deux clients différents.

27. Détecter les clients inactifs (**NOT EXISTS**)

Énoncé : Repérer les clients qui n'ont passé aucune commande depuis 6 mois.

```
SELECT cl.client_id, cl.nom
FROM clients cl
WHERE NOT EXISTS (
    SELECT 1 FROM commandes co WHERE co.client_id = cl.client_id
)
AND cl.date_inscription < CURRENT_DATE - INTERVAL '6 months';
```

Le piège : le filtre de date porte sur `clients.date_inscription`, pas sur la date de la dernière commande. La requête confond ancienneté du compte et inactivité réelle : un client inscrit depuis longtemps mais actif la semaine dernière peut être mal classé.

```
SELECT cl.client_id, cl.nom
FROM clients cl
WHERE NOT EXISTS (
    SELECT 1 FROM commandes co
    WHERE co.client_id = cl.client_id
    AND co.date_commande >= CURRENT_DATE - INTERVAL '6 months'
);
```

Erreur junior : appliquer le filtre de date sur la mauvaise table, au lieu de le mettre dans la condition du `NOT EXISTS` sur la date de commande.

28. Fusionner deux sources de commandes (**UNION**) PIÈGE CLASSIQUE

Énoncé : Fusionner les commandes actuelles et les commandes archivées de l'ancien système pour un rapport complet.

```
SELECT commande_id, client_id, montant_total FROM commandes
UNION
SELECT commande_id, client_id, montant_total FROM commandes_archive;
```

Le piège : `UNION` ne déduplique que les lignes strictement identiques sur les trois colonnes. Une commande migrée puis corrigée après coup (un montant réconcilié par la compta, par exemple) n'est plus identique des deux côtés : elle apparaît deux fois dans le rapport, une fois par table, avec le même `commande_id` mais un montant différent. Le rapport laisse croire à deux commandes, alors que c'est une seule commande dupliquée par la migration.

```
SELECT commande_id, client_id, montant_total FROM commandes
UNION ALL
SELECT ca.commande_id, ca.client_id, ca.montant_total
FROM commandes_archive ca
WHERE NOT EXISTS (
    SELECT 1 FROM commandes co WHERE co.commande_id = ca.commande_id
);
```

Erreur junior : compter sur UNION pour éliminer les doublons de migration, alors qu'elle ne détecte que des lignes identiques sur toutes les colonnes; il faut dédupliquer explicitement sur `commande_id` et garder la version de la table courante.

29. Sous-requête corrélée et agrégat imbriqué (mini-test) ÉLIMINATOIRE

Énoncé : Lister les clients dont le CA validé dépasse le CA moyen des clients, sans recalculer trois fois le même agrégat.

```
SELECT cl.client_id, cl.nom,
       (SELECT SUM(co.montant_total) FROM commandes co
        WHERE co.client_id = cl.client_id) AS ca_client
FROM clients cl
WHERE (SELECT SUM(co.montant_total) FROM commandes co
       WHERE co.client_id = cl.client_id)
      > (SELECT AVG(t.ca) FROM (
        SELECT client_id, SUM(montant_total) AS ca
        FROM commandes GROUP BY client_id) t);
```

Le piège : le même agrégat, le CA d'un client, est écrit trois fois, sans jamais filtrer `statut = 'validee'` comme l'énoncé le demande : le CA affiché est gonflé par les commandes annulées et remboursées. PostgreSQL ne mutualise pas non plus les deux sous-requêtes identiques du SELECT et du WHERE : il recalcule le total du client à chaque ligne, deux fois de suite, en plus de la moyenne globale.

```
WITH ca_par_client AS (
    SELECT client_id, SUM(montant_total) AS ca
    FROM commandes
    WHERE statut = 'validee'
    GROUP BY client_id
)
SELECT cl.client_id, cl.nom, cpc.ca
FROM clients cl
JOIN ca_par_client cpc ON cpc.client_id = cl.client_id
WHERE cpc.ca > (SELECT AVG(ca) FROM ca_par_client);
```

Erreur junior : répéter la même sous-requête d'agrégat plusieurs fois sans filtrer le statut, au lieu de la calculer une seule fois dans une CTE et de la réutiliser.

Dernière ligne droite : la requête 30 combine, volontairement, tous les pièges vus jusqu'ici.

30. La requête piège finale, celle qui combine tout ÉLIMINATOIRE

Énoncé : Produire le classement du CA validé du mois en cours par client, y compris les clients sans commande.

```
SELECT cl.client_id, cl.nom,
       SUM(co.montant_total) AS ca_mois,
       RANK() OVER (ORDER BY SUM(co.montant_total) DESC) AS rang
FROM clients cl
LEFT JOIN commandes co ON co.client_id = cl.client_id
WHERE co.statut = 'validee'
      AND DATE_TRUNC('month', co.date_commande)
          = DATE_TRUNC('month', CURRENT_DATE)
GROUP BY cl.client_id, cl.nom
ORDER BY rang;
```

Le piège : exactement comme à la requête 6, le WHERE sur co.statut et sur le mois filtre après la jointure : tous les clients sans commande validée ce mois-ci, y compris ceux sans aucune commande, disparaissent du résultat alors que l'énoncé demande explicitement de les inclure.

```
SELECT cl.client_id, cl.nom,
       COALESCE(SUM(co.montant_total), 0) AS ca_mois,
       RANK() OVER (ORDER BY COALESCE(SUM(co.montant_total), 0) DESC) AS rang
FROM clients cl
LEFT JOIN commandes co
  ON co.client_id = cl.client_id
  AND co.statut = 'validee'
  AND DATE_TRUNC('month', co.date_commande)
      = DATE_TRUNC('month', CURRENT_DATE)
GROUP BY cl.client_id, cl.nom
ORDER BY rang;
```

Erreur junior : filtrer sur co.statut et sur le mois dans le WHERE au lieu du ON, exactement le piège de la requête 6 ; une fois corrigé, les clients sans commande produisent un SUM NULL qu'il faut couvrir avec COALESCE.

La checklist de relecture avant de rendre un test SQL

Quatre questions à te poser avant de cliquer sur envoyer, dans cet ordre.

Périmètre filtré

Le bon statut (validée, annulée, remboursée) est-il posé avant, et non après, le premier agrégat ?

NULL et casse

Un WHERE, un NOT IN ou un tri peuvent-ils être cassés par une valeur NULL ?

Cardinalité

Une jointure peut-elle dupliquer des lignes avant un agrégat ?

Fenêtre et alias

Chaque OVER a-t-il son PARTITION BY et son ORDER BY explicites ?

30 sur 30. Tu n'as pas besoin de mémoriser ces requêtes au mot près. Ce qui compte, c'est le réflexe : vérifier la casse et les NULL avant un WHERE, contrôler la cardinalité avant une jointure, poser PARTITION BY avant une fenêtre. C'est cette posture, pas la syntaxe, qui distingue un candidat qu'on retient d'un candidat qui a mémorisé un cours.

Fait main, en solo.

Si ce corrigé t'a servi, partage-le : il peut débloquent un autre candidat avant son test. Et si un des 30 pièges t'a rappelé une vraie mésaventure, raconte-la en commentaire, c'est là que je pioche mes prochains guides.

Dylan

